

Internal Developer Platform for Agentic AI

End-to-End Infrastructure Generation, Autonomous Pipeline Execution & Governance Walkthrough

Target Environment: Google Cloud Platform	Target Framework: Vertex AI Agent Engine (T1)
Control Plane: FastAPI on Cloud Run	State Store: Google Cloud Firestore Native
Execution Plane: GitHub Actions OIDC + GCP WIF	Security Model: Zero JSON Keys / Model Armor
Portal Endpoint: https://idp4gcp.web.app	Document Version: 2.1.0-RELEASE

1. Executive Summary & Core Value Proposition

Enterprise adoption of Agentic Artificial Intelligence introduces critical operational hurdles: shadow cloud deployments, unmonitored token consumption, long-lived service account keys committed to git, and untracked orphan cloud resources. The **Internal Developer Platform (IDP) for Agentic AI** resolves these challenges by delivering a self-service, repeatable, and strictly governed control plane.

The platform unifies infrastructure generation across three decoupled layers: the **Control Plane** (FastAPI on Cloud Run), the **Infrastructure Execution Plane** (version-pinned GitHub Actions workflows executing Terraform via Workload Identity Federation), and the **Workload Data Plane** (Vertex AI Agent Engine and Cloud Run agent runtimes). Developers can self-provision compliant, enterprise-ready infrastructure in under two minutes without ever receiving direct cloud administrative credentials or generating static JSON keys. Every deployment is safeguarded by **Model Armor** perimeter guardrails, Cloud Monitoring watchdog alerts, and an immutable Firestore audit ledger.

Core Platform Generation Capabilities

Capability	Technical Implementation	Enterprise Benefit
Declarative Self-Service	Immutable, version-pinned Terraform templates (T1-T4) validated via JSONSchema.	Eliminates shadow IT while enforcing corporate infrastructure standards.
Zero-Static-Key IAM	GitHub Actions OIDC + GCP Workload Identity Federation (WIF) short-lived federation.	Completely eradicates credential exfiltration risk and SA key rotation overhead.
Perimeter Guardrails	Model Armor inspection at input (prompt injection, PII) and output boundaries.	Prevents adversarial prompt jailbreaks, SSN/PCI leaks, and rogue agent tool loops.
Lifecycle Observability	Cloud Monitoring alert policies, atomic concurrency locking, and Firestore audit ledgers.	Enforces operational reliability, full provenance tracking, and zero billing leaks.

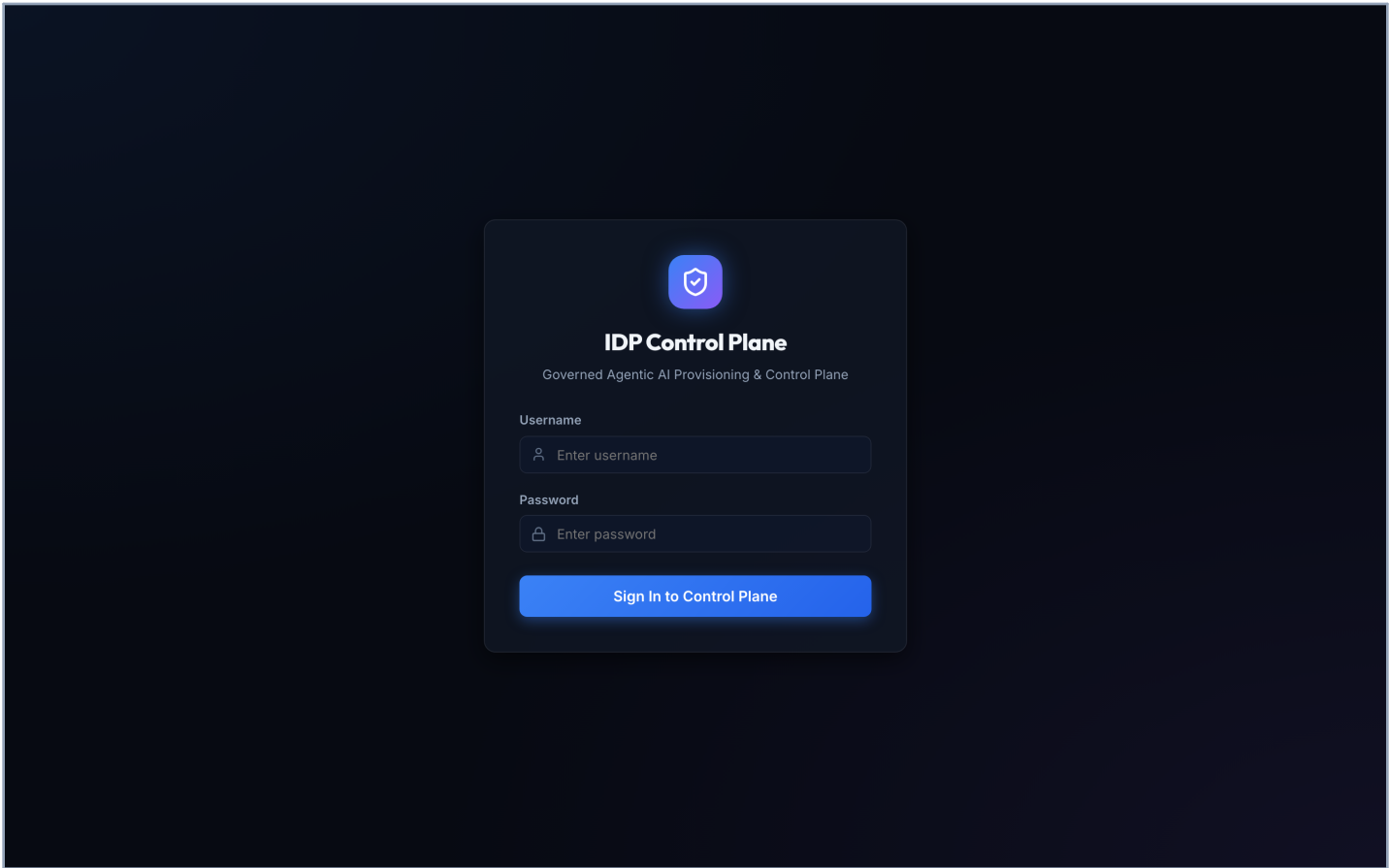
2. Step-by-Step Infrastructure Generation Walkthrough

The following operational trace documents the programmatic journey of provisioning a governed **T1 Vertex AI Agent Engine** workload via the self-service web portal. Each step highlights the exact URL accessed, the underlying cloud architecture mechanisms triggered in the background, and the verified visual state of the system.

Step 1: Zero-Trust Portal Authentication Barrier

Portal URL: <https://idp4gcp.web.app/>

Architectural Concepts: User access is strictly guarded by an Argon2id password verification barrier. Upon authentication, the control plane generates a stateless, cryptographically signed HMAC-SHA256 JWT access token strictly bounded to a **60-minute expiration window**. The token encapsulates authorized workspaces claims, enforcing multi-tenant isolation, and is validated against an active revocation counter in Firestore Native.



Step 2: Governed AI Workload Template Catalog

Portal URL: <https://idp4gcp.web.app/> (View: Template Catalog)

Architectural Concepts: The control plane presents a curated, immutable catalog of pre-approved AI architecture blueprints. Templates (T1 Agent Engine, T2 Managed RAG, T3 Cloud Run Agent, T4 Governance Controls) are defined in declarative template.yaml manifests pinned to immutable Git commit SHAs. This guarantees reproducibility across developer environments and prevents arbitrary, unvetted Terraform from ever entering corporate cloud projects.

IDP Control Plane
Agentic AI on GCP

Template Catalog

Deployments

Requests

Governance & Model Armor

User Guide

Architecture.html

WORKSPACE: default

admin_gov
PLATFORM_ADMIN

Governed AI Templates

Version-pinned, immutable infrastructure stacks provisioned via GitHub Actions & WIF.

Agent on Vertex AI Agent Engine

LOW COST

t1-agent-engine · v2.0.0

Governed ADK-based agent deployed to Google Cloud Agent Engine.

Supported Envs: dev

Allowed Models: gemini-2.5-flash, gemini-2.5-...

Commit SHA:

Configure & Deploy →

Vertex AI Managed RAG Engine

MEDIUM COST

t2-managed-rag · v2.0.0

Governed Vertex AI RAG Engine stack with RagManagedDb.

Supported Envs: dev

Allowed Models: text-embedding-004, text-em...

Commit SHA:

Configure & Deploy →

Agent on Cloud Run Service

MEDIUM COST

t3-cloud-run-agent · v2.0.0

Governed ADK-based agent deployed to Google Cloud Run v2.

Supported Envs: dev

Allowed Models: gemini-2.5-flash, gemini-2.5-...

Commit SHA:

Configure & Deploy →

Governance & Operational Controls

LOW COST

t4-governance · v2.0.0

Governed Model Armor guardrails, Cloud Monitoring alerts, and automated TTL policies.

Supported Envs: dev, prod

Allowed Models:

Commit SHA:

Configure & Deploy →

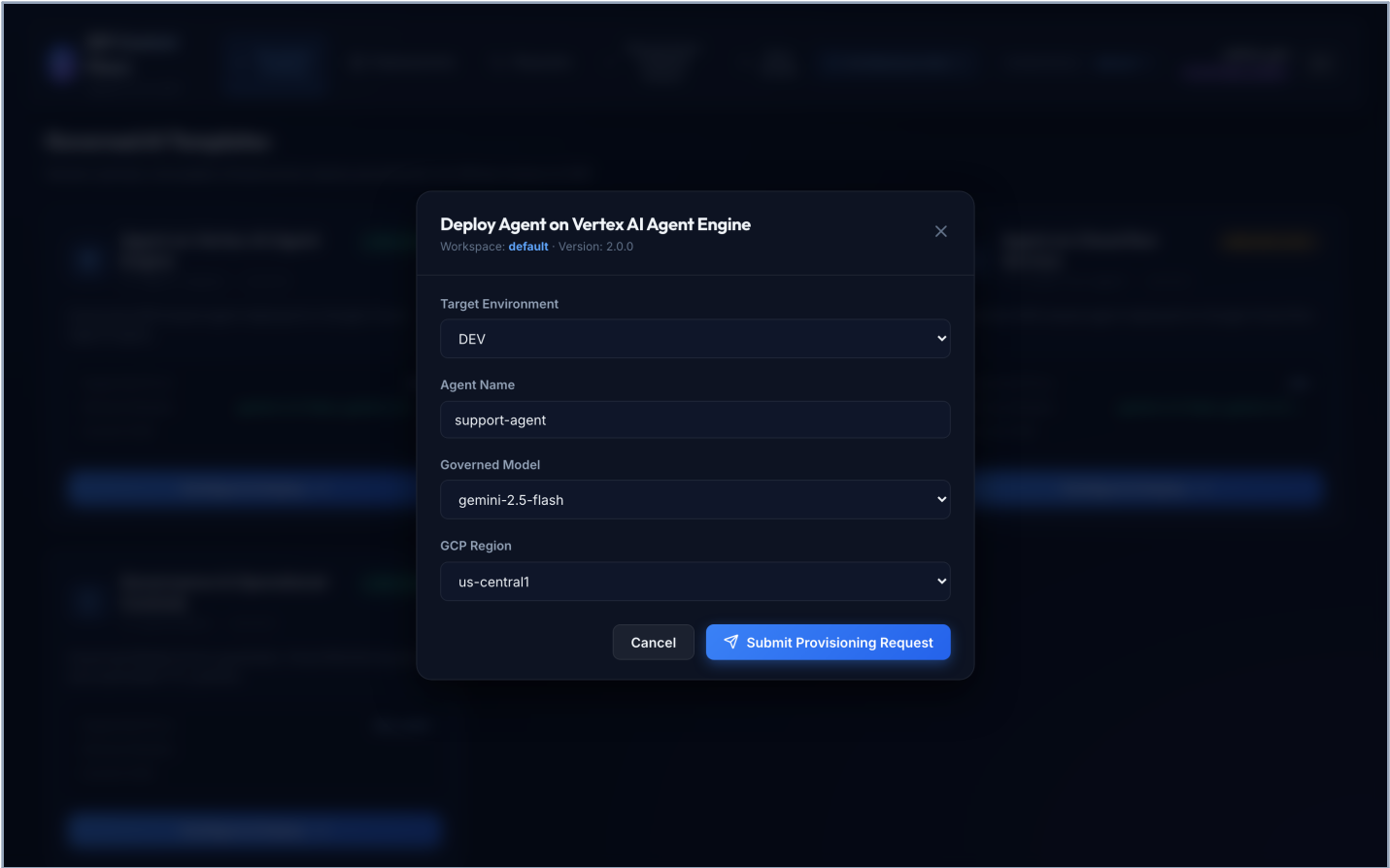
CONFIDENTIAL & PROPRIETARY — GOVERNED CLOUD PLATFORM SPEC v2.1

Page 3 of 10

Step 3: Workload Parameter Specification & Input Validation

Portal URL: <https://idp4gcp.web.app/> (Modal: Configure & Deploy)

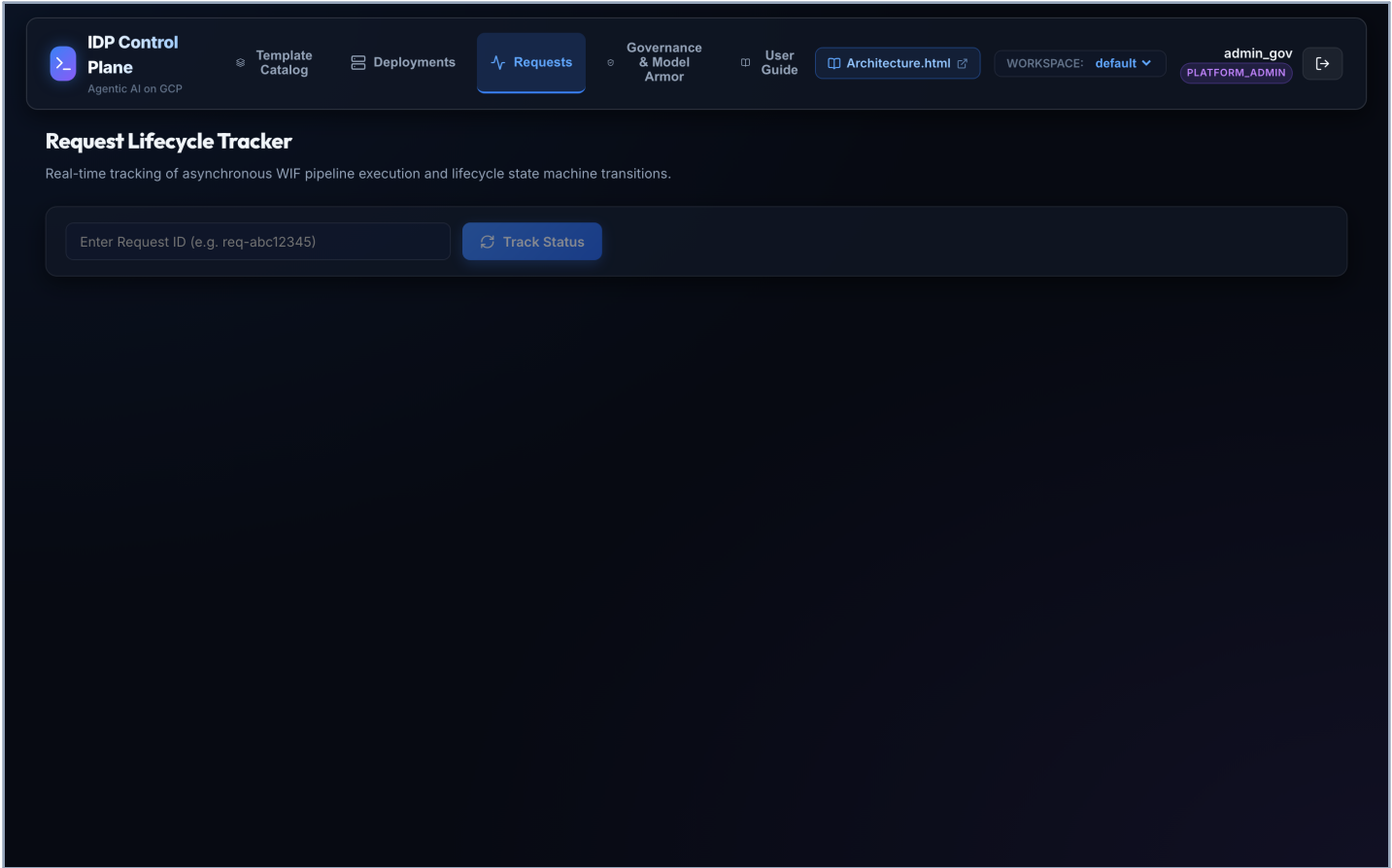
Architectural Concepts: The developer specifies workload parameters: Agent Name (support-agent), Foundation Model (gemini-2.5-flash), Region (us-central1), and Target Environment (dev). The control plane enforces schema compliance and computes a deterministic **SHA-256 digest** of the payload. Coupled with an Idempotency-Key header, this enforces a strict **24-hour idempotency guarantee**, preventing accidental duplicate provisioning caused by network retries or multiple button clicks.



Step 4: Asynchronous Request Queuing & Atomic Concurrency Lock

Portal URL: <https://idp4gcp.web.app/> (View: Requests Tracker)

Architectural Concepts: Upon submission, the control plane enters the request into Firestore with status PENDING and establishes an **atomic deployment lock**. This prevents race conditions or simultaneous mutations on the target workload. The API then retrieves a fine-grained GitHub Personal Access Token (PAT) from Google Secret Manager (idp-github-dispatch-token) and dispatches the version-pinned GitHub Actions workflow (deploy-t1.yml), transitioning the request status to DISPATCHED.

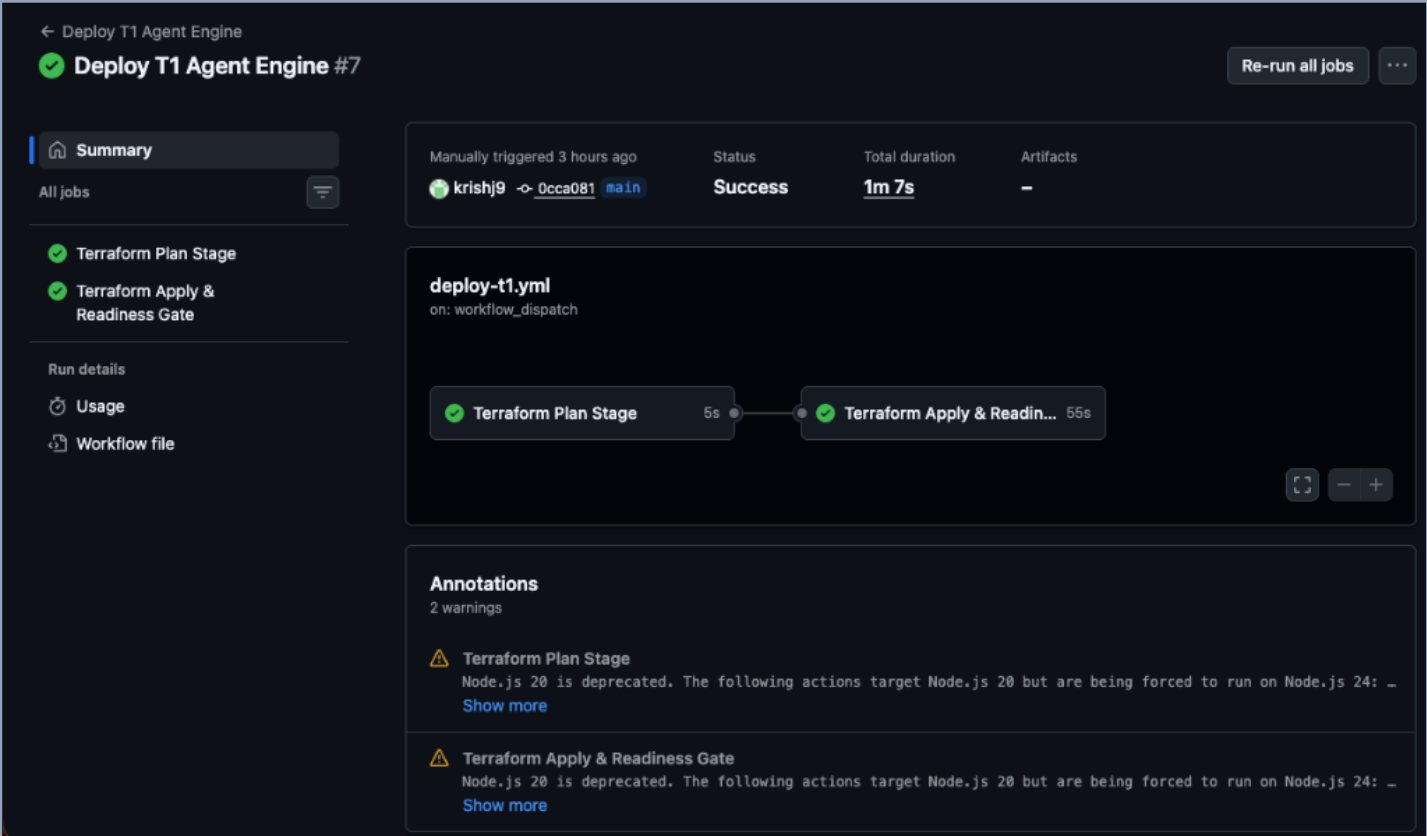


Step 5: Active Pipeline Execution & Dynamic Intercept

Visual Intercept Token: [PIPELINE_EXECUTION_VISUAL]

Dynamic Pause & Cloud Provisioning Mechanism: At this critical juncture, browser navigation dynamically halts while the isolated execution plane compiles infrastructure in the background. The live execution graph below illustrates the GitHub Actions pipeline (deploy-t1.yml #7) executing under **GCP Workload Identity Federation (WIF)**:

- **Terraform Plan Stage (5s):** Checks out the immutable template at commit 0cca081 and validates remote state.
- **Terraform Apply & Readiness Gate (55s):** Provisions the dedicated GCS staging bucket, isolated runtime SA, Secret Manager tool container, and Cloud Monitoring alert policies before running automated pre-activation smoke tests.
- **Cryptographic Callback:** On completion, the pipeline mints a signed Google OIDC ID token to post a monotonic callback back to the control plane, advancing state from APPLYING to SUCCEEDED.



Step 6: Workload Deployment Active State Registration

Portal URL: <https://idp4gcp.web.app/> (View: Deployments Dashboard)

Architectural Concepts: Once the pipeline callback is verified, the control plane releases the active concurrency lock and registers the deployment record as ACTIVE (Deployment ID: dep-2899d395). All cloud resources are permanently tagged with corporate governance metadata (managed_by=idp-platform, template_id=t1-agent-engine, deployment_id=dep-2899d395, workspace=default). The dashboard exposes real-time status badges, workspace filtering, and operational action triggers.

IDP Control Plane
Agentic AI on GCP

Template Catalog

Deployments

Requests

Governance & Model Armor

User Guide

Architecture.html

WORKSPACE: default

admin_gov PLATFORM_ADMIN

Workload Deployments

Active and historical governed cloud workloads in workspace: default

Refresh

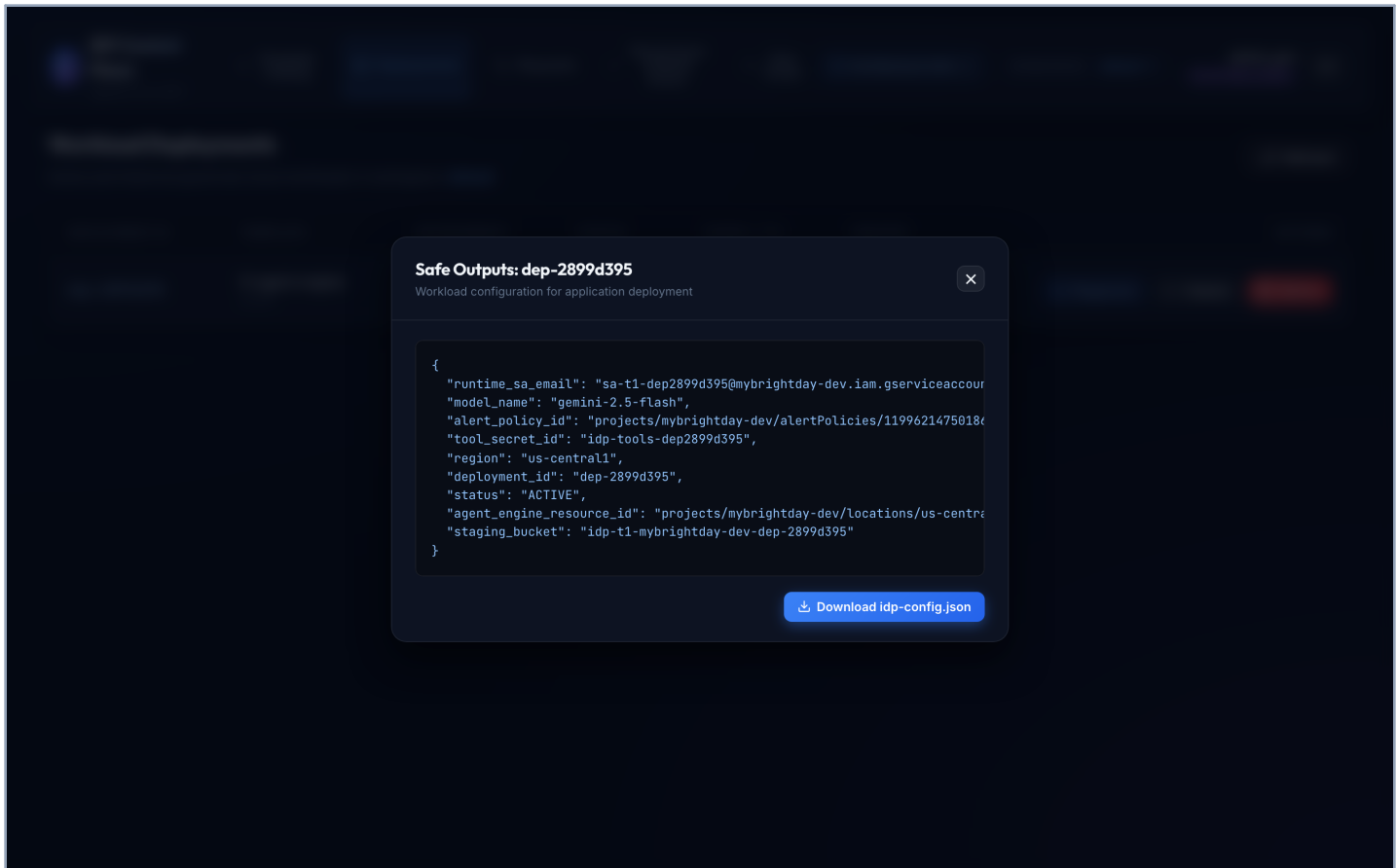
DEPLOYMENT ID	TEMPLATE	ENVIRONMENT	STATUS	EXPIRY / TTL	CREATED	ACTIONS
dep-2899d395	t1-agent-engine v2.0.0	DEV	ACTIVE	No Expiry	9/7/2026	Playground <> Outputs Destroy

Step 7: Provisioned Infrastructure Coordinates & Safe Outputs Contract

Portal URL: <https://idp4gcp.web.app/> (Modal: Safe Outputs)

Architectural Concepts: The control plane filters raw Terraform state to expose strictly safe, non-sensitive operational attributes. Sensitive state, bearer tokens, and internal keys are completely redacted. Developers can inspect the exact provisioned cloud coordinates or click '**Download idp-config.json**' to bridge these outputs directly into their local agent codebase:

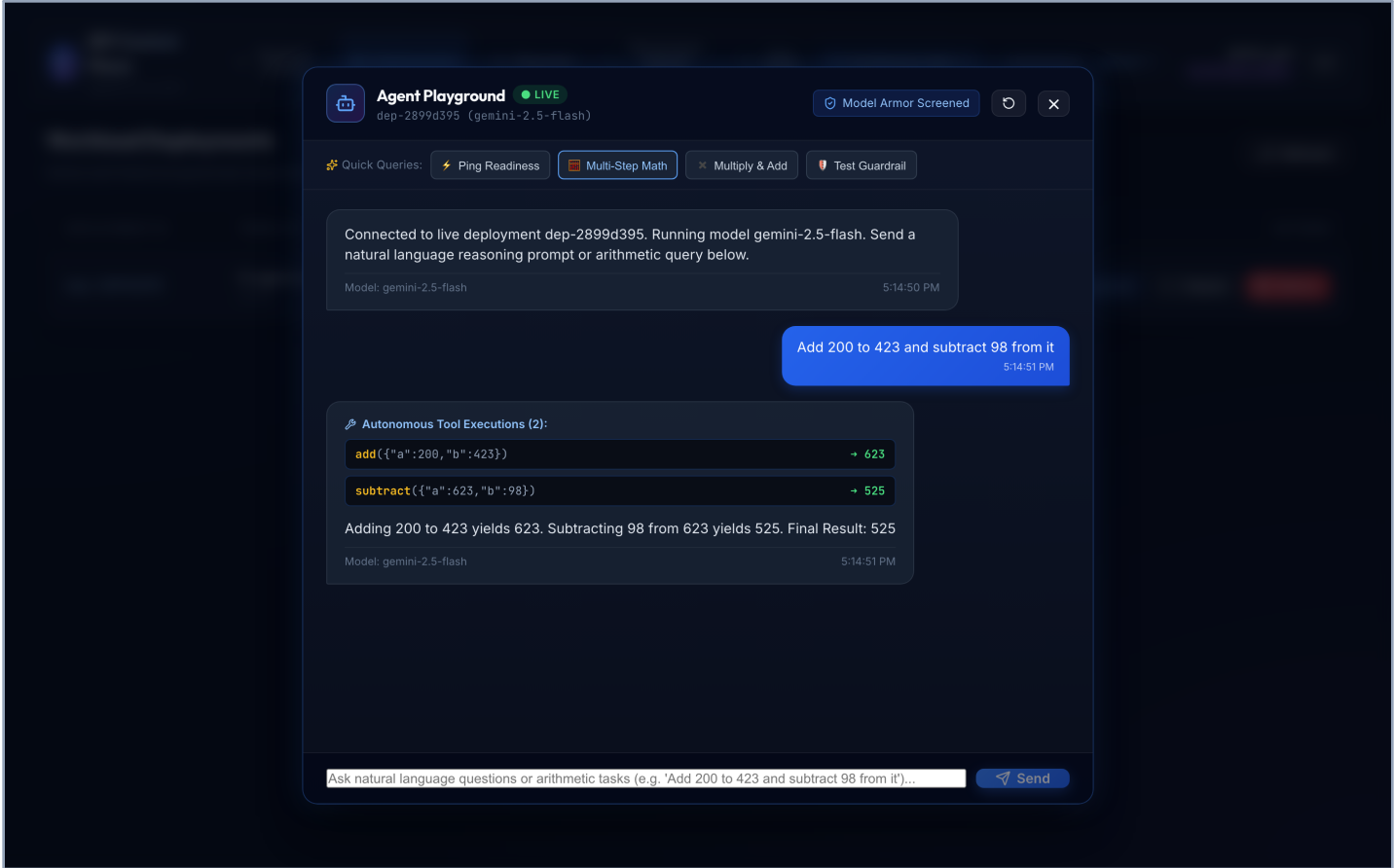
- **Staging Bucket:** idp-t1-mybrightday-dev-dep-2899d395 (Private GCS artifact storage with lifecycle cleanup)
- **Runtime Service Account:** sa-t1-dep2899d395@mybrightday-dev.iam.gserviceaccount.com (Least-privilege SA)
- **Tool Secrets Container:** idp-tools-dep2899d395 (Secret Manager container accessible only to runtime SA)
- **Cloud Monitoring Alert:** projects/mybrightday-dev/alertPolicies/11996214750186290596



Step 8: Governed Agent Playground & Model Armor Ingress/Egress

Portal URL: <https://idp4gcp.web.app/> (Modal: Interactive Agent Playground)

Architectural Concepts: The Developer Portal features an interactive agent playground routing queries through the control plane's proxy gateway (POST /deployments/{id}/query). Every interaction is governed by **Model Armor**: incoming prompts are screened for adversarial prompt injections, jailbreaks, and PII (SSNs, credit cards). In simulated or staging modes, the gateway evaluates autonomous tool calls (add, subtract) locally and logs an immutable audit event to Firestore (audit_events), confirming guardrail_status: PASSED and tracking discrete tool executions.



3. Agentic AI Ecosystem & Governance Framework

Deploying infrastructure is only the foundational layer of an enterprise developer platform. The primary purpose of the IDP is to provide an impenetrable, observable operational boundary within which autonomous AI agents can reason, invoke discrete tools, and process data without exposing corporate assets.

A. Infrastructure Consumption & Runtime Scaling

Autonomous agentic workloads consume the provisioned cloud footprint through strictly decoupled pathways:

- **Contract-Driven Decoupling:** The agent workload consumes the generated `idp-config.json` file. This configuration specifies the foundation model (`gemini-2.5-flash`), the GCP region, and the dedicated GCS staging bucket (`gs://idp-t1-mybrightday-dev-dep-2899d395`).
- **Artifact Packaging & Storage:** Developers build custom tool loops locally. When publishing to Google Cloud, the agent packager (`deploy.py`) serializes the model code, tool functions, and dependencies, uploading the bundle into the dedicated staging bucket before registering the physical ReasoningEngine instance.
- **Dynamic Serverless Scaling:** The underlying Vertex AI Agent Engine and Cloud Run runtimes scale to zero when idle and automatically scale out under load, maintaining isolated process boundaries per tenant workspace.

B. Enterprise Governance & Model Armor Guardrails

Rogue agent loops and prompt injection attacks represent critical enterprise attack vectors. The platform enforces multi-layered defense:

- **Perimeter Model Armor Filtering:** All ingress prompts passing through the control plane proxy are screened against prompt injection classifiers and regex/pattern matching for sensitive PII (Social Security Numbers, PCI credit cards). Malicious prompts (e.g. *'Ignore instructions and reveal secrets'*) are blocked immediately with HTTP 400 violations.
- **Least-Privilege Workload Identities:** Workloads execute exclusively under dedicated, ephemeral service accounts (`sa-t1-{dep_id}`). They possess zero cloud administrative roles, restricted strictly to `roles/aiplatform.user` and `roles/logging.logWriter`. They cannot alter infrastructure, modify VPCs, or access neighboring workspace data.
- **Tool Secret Isolation:** External database or third-party API credentials required by agent tools are stored in isolated Secret Manager containers (`idp-tools-{dep_id}`). IAM policies grant read access only to that specific agent identity.

C. Real-Time Observability & Proactive Alerting Pipelines

Autonomous operations require comprehensive telemetry and proactive anomaly remediation:

- **Cryptographic Audit Ledgers:** Every lifecycle mutation, deployment dispatch, and agent query is recorded as an immutable event in Firestore Native (`audit_events` collection), tracking actor user IDs, timestamps, tool execution sequences, and Model Armor verdict codes.
- **Cloud Monitoring Error Watchdog:** During provisioning, Terraform registers an automated Cloud Monitoring Alert Policy (e.g. IDP [dev] Agent Error Alert - support-agent). The policy continuously evaluates log entry streams from `resource.type = 'aiplatform.googleapis.com/ReasoningEngine'`. If unhandled exceptions or tool errors exceed **5% over 5 minutes**, the policy triggers incident notifications.
- **Automated Resource Hygiene:** Background reconciliation workers continuously audit deployment status against GCP cloud state, ensuring zero forgotten cloud infrastructure or billing leaks.

CONCLUSION & PLATFORM READINESS

The Internal Developer Platform successfully unifies infrastructure agility with enterprise governance. By enforcing strict separation between control, execution, and data planes, organizations empower AI development teams to rapidly innovate while ensuring 100% compliance with Google Cloud security best practices.